

FPGA-Based Real-Time Road Hazard Classification and Adaptive Indicator Control System

Internship Project Documentation

Submitted By

**ADLEE GEORGE CRASTO
NIKHIL NAGESH AMBIG**

Internship Organization

**VIVEQA
Anmaya Technologies**

Table of Contents

1. Project Overview
2. Project Objectives
3. Problem Statement
4. Existing System
5. Proposed System
6. Design and Architecture
7. Hardware Components
8. Software Requirements
9. Working Principle
10. System Flow
11. Implementation Approach
 - 11.1 Module Descriptions
 - 11.2 Build Instructions
 - 11.3 Run Instructions
 - 11.4 Testing Procedure
12. Test Results / Expected Results
13. Conclusion
14. References

1. Project Overview

This project presents an FPGA-Based Real-Time Road Hazard Classification and Adaptive Indicator Control System, designed to detect and respond to road surface conditions using a hardware finite state machine implemented on an Artix-7 FPGA. A Python script on a host PC emulates a road-condition sensor by encoding three possible road states — Normal, Speed Hump, and Pothole — as 4-byte pulse packets, which are transmitted to the FPGA over UART at 115200 baud through a USB-to-TTL adapter.

The FPGA receives the incoming bytes, classifies the road condition in hardware using a Verilog-based finite state machine (FSM), and drives two output indicators in real time: a set of three status LEDs and a MAX7219-based seven-segment display that shows the letter N (Normal), H (Hump) or P (Pothole). Whichever non-normal hazard is detected, the system holds that indication for 4 seconds before automatically restoring the Normal indication, without any external sensor or further host intervention, functioning as a self-clearing, time-bounded hazard alert.

2. Project Objectives

- Design an FPGA-based real-time road hazard classification system.
- Receive simulated road-condition data from a PC over UART at 115200 baud.
- Classify road conditions (Normal, Hump, Pothole) using a hardware threshold-based classifier.
- Implement a Finite State Machine to manage NORMAL, HUMP, POTHOLE and RECOVERY states.
- Provide visual feedback through status LED indicators for each road condition.
- Display the real-time classification result on a MAX7219 seven-segment display.
- Implement an automatic 4-second recovery mechanism that restores the Normal state without external intervention.
- Demonstrate complete FPGA-based digital design using Verilog HDL, including UART reception, packet classification, FSM control, and SPI-based display driving.

3. Problem Statement

Conventional road-condition feedback in vehicles is largely manual or absent: drivers typically have no real-time indication of an approaching speed hump or pothole beyond what they can see or feel, and there is no automatic, low-latency mechanism that classifies a hazard and clears itself once the hazard has passed. This project addresses that gap by implementing an FPGA-based hazard classification system that responds to incoming sensor-style data within a fixed clock-cycle latency, displays the classification result immediately, and reverts to the default Normal indication automatically after a defined recovery window, all without software intervention once configured.

4. Existing System

Existing road-hazard indication approaches are typically either purely passive (relying on the driver's own observation) or implemented in software on general-purpose microcontrollers, which introduces variable response latency, depends on an operating system or interrupt scheduler, and offers no inherent guarantee of a fixed, deterministic recovery period. Such systems also typically require the indication to be cleared manually or by a separate timer routine, rather than being built into the control logic itself.

5. Proposed System

The proposed system uses an Artix-7 FPGA (XC7A35T-FTG256-1) as the central hardware controller. A Python script on the host PC encodes the road condition to be simulated as a 4-byte pulse packet and transmits it over UART. A hardware UART receiver recovers each byte, and a dedicated classifier module buffers every 4 bytes into a packet and determines the road class by comparing the packet's peak byte value against two fixed thresholds. A security-style finite state machine then evaluates the classification result: a NORMAL classification keeps the system idle, while a HUMP or POTHOLE classification moves the FSM into the corresponding hazard state, drives the appropriate LED and 7-segment indication, and starts a 4-second hardware timer. On expiry of the timer the FSM automatically transitions through a RECOVERY state back to NORMAL, restoring the default indication without any further input.

6. Design and Architecture

The system follows a pipeline architecture: a UART receiver recovers bytes from the serial line, a classifier groups every four bytes into one packet and derives a road class, an FSM turns that class into a stable, time-bounded system state, and a display driver renders the current state on the MAX7219 module. Status LEDs are derived combinationally from the same FSM output, so the LEDs and the 7-segment character are always consistent with one another.

Signal flow: Python (PC) → UART TX → uart_rx (serial-to-parallel, 1 byte) → hazard_classifier (4-byte packet buffer & threshold classification) → fsm_controller (state machine, 4 s recovery timer) → seg_display (SPI driver for MAX7219) and on-board status LEDs.

Block Responsibilities

Block	Responsibility
uart_rx	Synchronises the asynchronous RX line into the FPGA clock domain, detects the start bit, samples 8 data bits at the bit-centre, and pulses rx_valid for one clock when a byte is complete.
hazard_classifier	Accumulates four received bytes into a packet, tracks the peak byte value seen, and on the fourth byte compares the peak against two thresholds to decide NORMAL / HUMP / POTHOLE, pulsing class_valid for one clock.
fsm_controller	Holds the system in NORMAL, HUMP, POTHOLE or RECOVERY, applies hazard upgrade/refresh rules, drives a display-selection code and a speed-selection code, and times the 4-second auto-recovery.
seg_display	Generates a 1 MHz SPI tick from the 24 MHz clock and bit-bangs MAX7219 initialisation and digit-write commands so all four digits show the pattern selected by display_sel.
road_hazard_top	Top-level wrapper that instantiates the four blocks above, wires them together, and derives the three status LEDs combinationally from display_sel.

7. Hardware Components

- Artix-7 FPGA Development Board (Xilinx XC7A35T-FTG256-1), board AT-STLN-ARTIX7-001.
- USB-to-TTL Adapter (CH340 or CP2102) for 3.3 V UART communication.
- MAX7219-driven 4-digit Seven-Segment Display Module (on-board).

- 3x Status LEDs (FPGA Bank 35) used as Normal / Hump / Pothole indicators.
- Onboard 24 MHz Clock Oscillator (pin D13).
- 15 V DC barrel-jack power supply (centre-positive).
- USB-C cable for the FT232H J1 programming port.
- PC running Vivado 2025.2 and Python 3.x for the sensor-emulation script.

Note: the documented hardware design also defines an SG90 servo output (PMOD J16 Pin 3 / pin R3) for a position-based indicator, in addition to the LED and display outputs. The Verilog top module covered in this documentation is the LED + display variant; see Section 11.1 and the closing note for the servo-integration status.

8. Software Requirements

- Vivado Design Suite 2025.2
- Verilog HDL
- Python 3.x (PySerial) for the UART sensor-emulation script

9. Working Principle

The host PC sends a 4-byte packet representing the road condition to be simulated. The FPGA's UART receiver captures each byte and the hazard classifier buffers four bytes at a time, comparing the packet's peak value against fixed thresholds ($HUMP_THRESH = 20$, $POTHOLE_THRESH = 100$) to decide the road class.

Normal road ($peak \leq 20$):

- FSM remains in / returns to the NORMAL state.
- Seven-segment display shows "N".
- Status LED0 is ON; LED1 and LED2 are OFF.

Speed hump ($20 < peak \leq 100$):

- FSM transitions to the HUMP state.
- Seven-segment display shows "H".
- Status LED1 is ON; LED0 and LED2 are OFF.
- After 4 seconds with no further hazard packet, the FSM automatically returns to NORMAL.

Pothole ($peak > 100$):

- FSM transitions to the POTHOLE state (a hump in progress is automatically upgraded to pothole).
- Seven-segment display shows "P".
- Status LED2 is ON; LED0 and LED1 are OFF.
- Any further non-normal packet received while still in POTHOLE restarts the 4-second timer, extending the alert; after 4 seconds with no further hazard packet, the FSM returns to NORMAL.

10. System Flow

POWER ON / CONFIGURE FPGA → WAIT FOR UART PACKET (4 bytes) → CLASSIFY PEAK VALUE →
PEAK $\leq$ 20? — YES → NORMAL: LED0 ON, display "N" → remain idle awaiting next packet.

PEAK $\leq$ 20? — NO, 20 < PEAK $\leq$ 100 → HUMP: LED1 ON, display "H" → start 4 s timer → new POTHOLE packet arrives? YES → upgrade to POTHOLE state, restart timer; NO → timer expires → RECOVERY → NORMAL.

PEAK > 100 → POTHOLE: LED2 ON, display "P" → start 4 s timer → further non-normal packet arrives? YES → restart timer, remain in POTHOLE; NO → timer expires → RECOVERY → NORMAL.

Notes: Normal packet (peak $\leq$ 20) → NORMAL, LED0, display N. Hump packet → HUMP, LED1, display H, 4 s auto-recovery, upgradable to POTHOLE. Pothole packet → POTHOLE, LED2, display P, 4 s auto-recovery, re-extendable by further hazard packets.

11. Implementation Approach

The system was implemented in Verilog HDL on Vivado 2025.2, targeting the Xilinx XC7A35T-FTG256-1 Artix-7 FPGA on the AT-STLN-ARTIX7-001 board. The design follows a modular approach in which UART reception, packet classification, finite state machine control and display driving are each implemented as independent, reusable Verilog modules and integrated under a single top-level module, `road_hazard_top.v`.

11.1 Module Descriptions

1. `uart_rx.v`

Serial-to-parallel UART receiver implementing the classic IDLE/START/DATA/STOP states, parameterised by `CLK_PER_BIT` = 208 (24,000,000 / 115200 $\approx$ 208 cycles per bit). Uses a 2-flip-flop synchroniser on the incoming RX line and samples each bit at its mid-point, shifting bits in LSB-first and pulsing `rx_valid` for one clock once a byte is confirmed by a valid stop bit.

2. `hazard_classifier.v`

Buffers every 4 received bytes as one packet, tracks the running peak (maximum) byte value, and on the fourth byte compares the peak against `HUMP_THRESH` (20) and `POTHOLE_THRESH` (100) to register `road_class`, pulsing `class_valid` for one clock. The POTHOLE threshold is checked first, so any packet whose peak exceeds 100 is classified POTHOLE.

3. `fsm_controller.v`

Core finite state machine with four states — NORMAL, HUMP, POTHOLE and RECOVERY. Uses a 32-bit counter to implement an exact 4-second timer (`DELAY_COUNTS` = 96,000,000 cycles at 24 MHz). Implements hazard-upgrade logic (a pothole detected during HUMP immediately escalates the state) and hazard-refresh logic (further hazard packets during POTHOLE restart the timer without changing state). Drives `display_sel` for the seven-segment driver and `speed_sel` for a future actuator stage.

4. `seg_display.v`

Drives the on-board MAX7219 seven-segment module over a bit-banged SPI-style interface to show "N", "H" or "P" across all four digits depending on `display_sel`. Derives a 1 MHz SPI tick from the 24 MHz system clock and sequences the standard MAX7219 initialisation commands (shutdown off, display-test off, decode-mode off, intensity, scan-limit) before continuously refreshing the digit registers.

5. `road_hazard_top.v`

Top-level module that integrates the UART receiver, hazard classifier, FSM controller and display driver into the complete classification system, and derives the three status LEDs combinationally from `display_sel` so the LEDs and the display character always agree.

6. road_hazard_top.xdc

Constraints file assigning LVCMOS33 I/O standard to every port, a 41.667 ns (24 MHz) clock period to clk_24mhz, and a false-path constraint on the asynchronous uart_rx input.

Complete Pin Assignments

Signal	FPGA Pin	Bank	Connected to	Direction
clk_24mhz	D13	15	24 MHz on-board oscillator	Input
uart_rx	T2	34	PMOD J16 Pin 1 (USB-TTL adapter)	Input
seg_din	J15	15	MAX7219 DIN (on-board)	Output
seg_cs (LOAD)	J16	15	MAX7219 LOAD (on-board)	Output
seg_clk	H12	15	MAX7219 CLK (on-board)	Output
led[0]	D5	35	Normal state indicator LED	Output
led[1]	A3	35	Hump state indicator LED	Output
led[2]	B4	35	Pothole state indicator LED	Output

11.2 Build Instructions

Step	Action
1	Open Vivado Design Suite and create a new RTL project targeting XC7A35T-FTG256-1.
2	Add all Verilog source files: road_hazard_top.v, uart_rx.v, hazard_classifier.v, fsm_controller.v, seg_display.v.
3	Add the road_hazard_top.xdc constraints file for pin and clock assignments.
4	Set road_hazard_top as the top module.
5	Run Synthesis.
6	Run Implementation.
7	Generate Bitstream.
8	Open Hardware Manager, Auto Connect, and Program the FPGA with the generated .bit file.

11.3 Run Instructions

Step	Action
1	Set the configuration mode switches before programming: SW17 (M2) = OFF, SW36 (M1) = OFF, SW37 (M0) = ON.
2	Power the board with the 15 V DC adapter; confirm all power LEDs glow.
3	Connect the USB-to-TTL adapter TX line to PMOD J16 Pin 1 (T2) and GND to PMOD J16 Pin 19; leave the adapter's VCC unconnected.

Step	Action
4	Program the FPGA via the USB-C / J1 port using the generated bitstream.
5	On the PC, run the Python sensor-emulation script and select the COM port assigned to the CH340/CP2102 adapter at 115200 baud.
6	Send the Normal, Hump, or Pothole packet (N / H / P key + Enter) and observe the seven-segment display and status LEDs update accordingly.
7	Confirm that, after 4 seconds with no further hazard packet, the display and LEDs automatically return to the Normal indication.

11.4 Testing Procedure

Test Case	Description	Expected Result	Status
TC-01	Normal road packet (peak $\leq$ 20)	Display shows "N", LED0 ON, LED1/LED2 OFF	Pass
TC-02	Speed hump packet (20 < peak $\leq$ 100)	Display shows "H", LED1 ON, LED0/LED2 OFF, 4 s timer starts	Pass
TC-03	Pothole packet (peak > 100)	Display shows "P", LED2 ON, LED0/LED1 OFF, 4 s timer starts	Pass
TC-04	Pothole packet received while in HUMP state	FSM upgrades immediately to POTHOLE and restarts the 4 s timer	Pass
TC-05	Auto-recovery after timeout	After 4 s with no further hazard packet, system returns to NORMAL ("N", LED0 ON)	Pass

12. Test Results

The FPGA-based system correctly classified all three simulated road conditions transmitted over UART. A Normal packet kept the display on "N" with LED0 active; a Hump packet switched the display to "H" with LED1 active; and a Pothole packet switched the display to "P" with LED2 active. In every hazard case, the system automatically reverted to the Normal indication exactly 4 seconds after the last hazard packet, with no host intervention required. A pothole packet received while a hump indication was active correctly upgraded the system directly to the Pothole state and restarted the recovery timer, confirming the hazard-escalation logic in fsm_controller.v.

Condition	7-segment	LED0	LED1	LED2
Power-on / default	N	ON	OFF	OFF
Python sends Normal packet	N	ON	OFF	OFF
Python sends Hump packet	H	OFF	ON	OFF
Python sends Pothole packet	P	OFF	OFF	ON
4 seconds after Hump/Pothole (no new packet)	N	ON	OFF	OFF

13. Conclusion

The FPGA-Based Real-Time Road Hazard Classification and Adaptive Indicator Control System successfully combines UART-based sensor emulation, threshold-based packet classification, finite-state-machine-controlled NORMAL/HUMP/POTHOLE/RECOVERY logic, and seven-segment display feedback into a reliable, self-clearing hazard-alert solution. The project demonstrates effective FPGA hardware design using Verilog HDL and highlights how a deterministic, hardware-timed recovery mechanism can provide consistent, low-latency hazard indication compared to software-driven alternatives, making it a suitable building block for vehicle-mounted or roadside hazard-warning systems.

14. References

- Xilinx Vivado Design Suite Documentation
- Artix-7 FPGA User Manual
- AT-STLN-ARTIX7-001 Hardware Reference Manual
- MAX7219 Seven-Segment Display Driver Datasheet
- Verilog HDL Reference Manual